1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.
Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

Register R0

00	00	00	12

31 16 15 0

Register R1

ff	ff	ff	80

31 16 15 0

Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

Adresse der Konstanten:

12	34	56	78

31 16 15 0

Inhalt Register R2 nach Anw-03:

20	00	00	20

31 16 15 0

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

ldr r2, [pc, #56]

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss
Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

Adresse v. VariableA (Reg. R0 nach Anw-05)

20	00	00	0C

31 16 15 0

Inhalt Register R1 nach Anw-05:

00	00	12	34

31 16 15 0

Anzahl von Anw-05 geladener Bits:

16

1x halb Wort
2 x bytes

Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:

47	11	12	34

31 16 15 0

Anzahl von Anw-06 geladener Bits:

32

Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

Adresse v. VariableC:

20	00	00	10

31 16 15 0

Inhalt von VariableC nach Anw-07:

34	12	11	47

31 16 15 0

Anzahl von Anw-07 gespeicherter Bits:

32

3 *Zugriff auf Felder*

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

	Anfangsadresse:	Anzahl Elemente:	Anzahl Bytes pro Element:
MeinHalbwortFeld:	20 00 00 14 (31 … 16 15 … 0)	6	4
MeinWortFeld:	20 00 00 20 (31 … 16 15 … 0)	6	8
MeinTextFeld:	20 00 00 38 (31 … 16 15 … 0)	2	1

Anw-08 *Anw-09* *Anw-10* Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für das jeweilige Element erfolgen.

Register R1 nach Anw-09:	Register R2 nach Anw-10:
00 00 00 22 (31 … 16 15 … 0)	00 00 00 3e (31 … 16 15 … 0)

Anw-11 *Anw-12* Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:

Register R4 nach Anw-12:		Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements:
00 00 00 45 (31 … 16 15 … 0)	Offset von 0x0a	10

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13	Register R5 nach Anw-13:	Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements:
20 00 00 16 (31 … 16 15 … 0)	00 00 00 3e (31 … 16 15 … 0)	2

Anw-14

Register R0 nach Anw-14	Register R6 nach Anw-14:	Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements:
20 00 00 18 (31 … 16 15 … 0)	00 00 ff cc (31 … 16 15 … 0)	4

Anw-15

Register R0 nach Anw-15	Adresse des von Anw-15 überschriebenen Elements:	Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements:
00 00 00 1a (31 … 16 15 … 0)	00 00 00 1a (31 … 16 15 … 0)	6

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

d 13 a 10
e 14 b 11
f 15 c 12

Höchstes Bit 0

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Anw-16	Reg. R1: 0001001001101010101010110111000	305419896	305419896
Anw-17	Reg. R2: 1001101110011010101001101000110	-1647683194	2647284102
Anw-18	Reg. R3: 1010111111111110101011111111110	-1342263298	2952703998
Anw-19	Ergebnis: ~~Ok~~ Nicht Ok	Ergebnis: ~~Ok~~ Nicht Ok	

31 … 16 15 … 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Anw-20	Reg. R4: 1100110000000000000000000000000	-872415232	3422552064
Anw-21	Reg. R5: 0100111000000000000000000000000	1308622848	1308622848
Anw-22	Reg. R6: 0111110000000000000000000000000	2113929216	2113929216
	Ergebnis: Ok ~~Nicht Ok~~	Ergebnis: ~~Ok~~ Nicht Ok	

31 … 16 15 … 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Anw-23	Reg. R7: 0010011100000000000000000000000	654311424	654311424
Anw-24	Reg. R8: 0100010100000000000000000000000	1157627904	1157627904
Anw-25	Reg. R9: 1110000100000000000000000000000	-503316480	3791650816
	Ergebnis: ~~Ok~~ Nicht Ok	Ergebnis: Ok ~~Nicht Ok~~	

31 … 16 15 … 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N Z